

ModularML: A Backend-Agnostic, Declarative Framework for Reproducible and Modular Machine Learning Experiments

Ben Nowacki¹, Tingkai Li¹, Mohammad Mundiwala¹, Hui Hua¹, Fei Miao¹, and Chao Hu¹

¹ University of Connecticut, United States ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Modern machine learning (ML), particularly deep learning, has become central to scientific research across domains such as energy systems, materials science, structural health monitoring, and biomedical engineering. However, core ML frameworks primarily focus on model implementation and optimization, leaving experiment structure, data partitioning, and reproducibility patterns to user-defined scripts. As a result, domain researchers often rely on fragmented scripts, backend-specific conventions, and ad hoc experiment management workflows that limit reproducibility and collaboration.

ModularML is an open-source Python framework designed to address this gap. It provides a backend-agnostic, declarative, and fully serializable architecture for ML experimentation, enabling users to define complete workflows—including data curation, feature processing, model architecture, training strategy, loss configuration, and evaluation—using modular, composable components that serialize into a single reproducible artifact.

At its core, ModularML introduces an abstraction for structured, backend-agnostic data handling and splitting; a directed acyclic graph (DAG) of model nodes with support for shared or node-specific optimization; multi-phase training orchestration with configurable freeze/unfreeze policies and phase- and node-specific loss definitions; and full pipeline serialization and visualization for transparent sharing and inspection of experiments. By separating experiment definition from backend implementation, ModularML allows researchers to collaborate and audit ML studies without requiring expertise in PyTorch (?), Keras ([Chollet & others, 2015](#)), or other specific ML libraries.

Statement of need

ML workflows in academia differ substantially from typical industry-focused ML development. Scientific researchers emphasize experimentation with domain-specific feature selection, sampling, and model architecture. Existing libraries such as PyTorch Lightning ([W. Falcon & The PyTorch Lightning team, 2019](#)), Catalyst ([Kolesnikov, 2018](#)), FastAI ([Howard & others, 2018](#)), and PyTorch Ignite ([Fomin et al., 2020](#)) can streamline training loops, but typically remain tied to a single backend, prioritize development productivity over experimentation standardization, and depend on implicit code-based configurations with limited traceability of dataset splitting.

In contrast, ModularML is built around the principle of *configuration-as-contract*: the entire ML experiment is represented as a structured, inspectable object. This design offers three major benefits:

1. **Reproducibility through serialization.** Entire experiments, from initial data curation to multi-stage training and evaluation, can be saved and shared as a single file. A collaborator can load this artifact and inspect the full pipeline without having to trace the source code.
2. **Backend abstraction.** Model nodes are agnostic to the backend in which the underlying model is defined. ModularML does not replace existing ML libraries such as PyTorch, Keras, or scikit-learn (Pedregosa et al., 2011); rather, it improves their usage by providing a unified interface that researchers can intuitively understand. This backend-agnostic design supports reuse of collaborator models regardless of which library was used to implement them.
3. **Transparency for non-ML experts.** Built-in summary and visualize methods for feature sets, model graphs, and experiment execution phases allow users and code reviewers to quickly inspect data splits, model architecture, loss routing, and training sequencing without needing to understand backend-specific implementation details.

These features directly address common reproducibility and collaboration challenges in research, while still supporting a comprehensive suite of modeling techniques and existing ML libraries.

State of the field

General-purpose deep learning frameworks (e.g., PyTorch, Keras) streamline model training but typically assume homogeneous backends and single-phase workflows. Data-centric libraries (e.g., Apache Arrow ("Arrow," n.d.), Hugging Face (Lhoest et al., 2021)) manage datasets but leave sampling and experiment scheduling to custom user code. Experiment managers (e.g., MLflow (Zaharia et al., 2018), Weights & Biases ("Wandb," n.d.)) track runs but do not define how data flows through modular graphs. ModularML sits between these layers. It provides concrete abstractions for data (`modularml.FeatureSet`), sampling logic (`modularml.Sampler`), modeling (`modularml.ModelGraph` composed of `modularml.ModelNodes`), losses (`modularml.AppliedLoss`), and orchestration (`modularml.ExperimentPh`) while remaining backend-agnostic and lightweight enough for research scripts or notebooks.

Software design

The ModularML architecture provides abstractions for data storage and processing, graph-based modeling, and execution of distinct training sequences (Figure 1). Rather than embedding these responsibilities within custom training scripts, ModularML separates them into explicit, composable components that can be independently configured, inspected, and serialized. This separation enables rapid experimentation with data pipelines, model architectures, and training strategies while preserving reproducibility and traceability across the entire machine learning workflow.

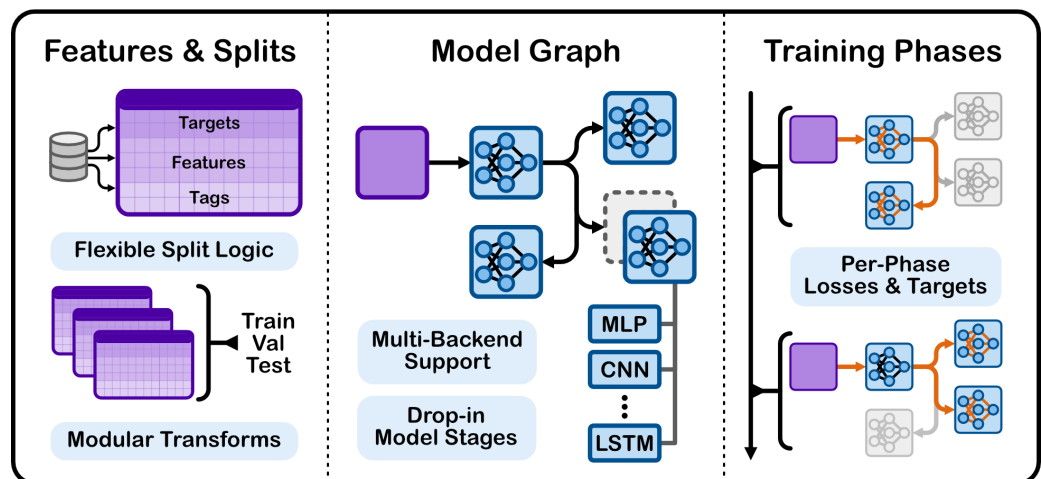


Figure 1: ModularML architecture overview: FeatureSet curation into samples with feature, target, and tag domains; flexible ModelGraph construction for rapid experimentation of model topology; and multi-phase training workflows sequenced within a single Experiment container.

FeatureSets

A `modularml.FeatureSet` organizes data into three intent-driven domains—features, targets, and tags—reflecting the core design philosophy of the data abstraction. Features represent model inputs (what the model learns from), targets represent model outputs (what the model is trained to predict or reproduce), and tags store optional metadata associated with each sample. Each sample, defined by its feature, target, and tag attributes, is assigned a globally unique identifier to ensure explicit traceability throughout the entire lifecycle of an experiment. This identifier propagates through splitting, sampling, batching, model execution, and evaluation, enabling transparent lineage tracking and reproducible analysis.

Data within a `modularml.FeatureSet` is stored in backend-agnostic containers supporting NumPy (Harris et al., 2020) arrays, Pandas (team, 2020) dataframes, PyTorch tensors, and TensorFlow (Abadi et al., 2015) tensors. All downstream operations, such as splitting, subsetting, and batching, operate using no-copy views of the underlying data. This design makes split definitions explicit and inspectable, reduces memory overhead, and minimizes the risk of data leakage or unintended experimentation bias.

Sampling

All subclasses of `modularml.Sampler` consume a `modularml.FeatureSet`, or subset views of one, and emit aligned batches, supporting stratification, grouping, and multi-role sampling needed for contrastive or paired training schemes. By separating sampling logic from model execution, ModularML makes data selection strategies explicit and reproducible rather than embedding them within custom training loops. This design allows researchers to experiment with different batching and sampling approaches without modifying model or training code.

ModelGraph

The full experiment model is represented as a DAG of interconnected `modularml.ModelNodes`. Each node wraps a user-defined or built-in ML model and exposes standardized interfaces for construction, optimization, and freezing. Specialized node types, such as `modularml.MergeNode`, extend this functionality by supporting multiple inputs and configurable merge operations including concatenation, aggregation, and padding. Together, these nodes form a `modularml.ModelGraph`, which manages dependency resolution and executes forward and backward passes in topological order. This graph-based design

105 separates model topology from training logic, enabling rapid experimentation with branching
106 architectures, multi-stage pipelines, and hybrid systems that combine learned and classical ML
107 components.

108 Losses and phases

109 `modularml.AppliedLoss` binds objectives to specific nodes within a `modularml.ModelGraph`,
110 enabling composite-loss training and targeted loss aggregation across different portions of
111 a model. These losses are tracked throughout execution, providing detailed records for
112 post-training analysis and comparison of experimental configurations.

113 Model execution is organized into a sequence of `modularml.ExperimentPhase` objects. Each
114 phase defines a reproducible unit of execution, including data sampling, loss definitions,
115 callbacks, and training or evaluation behavior. Built-in phase types include iterative training
116 (`modularml.TrainPhase`), single-pass fitting workflows (`modularml.FitPhase`), and inference-
117 only evaluation (`modularml.EvalPhase`), providing a consistent interface for constructing
118 complex multi-phase ML workflows.

119 Experiment orchestration

120 The `modularml.Experiment` class serves as the top-level container for a ML workflow, binding
121 together one or more `modularml.FeatureSets`, a `modularml.ModelGraph`, and a sequence of
122 execution phases. By explicitly separating experiment definition from execution, ModularML
123 supports workflows such as pretraining, fine-tuning, and evaluation while retaining all
124 information required to reproduce each stage. The `modularml.Experiment` also provides
125 serialization, checkpointing, and execution tracking, allowing complete experiments to be
126 exported, shared, and reloaded as self-contained artifacts.

127 Research impact statement

128 ModularML provides structured abstractions for scientists prototyping hybrid ML systems that
129 combine learned encoders, classical regressors/classifiers, and domain-specific samplers. By
130 guaranteeing that every component can be serialized, checkpointed, and replayed, the framework
131 supports reproducible experiments and facilitates sharing of trained graphs or datasets between
132 collaborators. Its backend-agnostic graph execution simplifies comparisons across PyTorch,
133 TensorFlow, and scikit-learn implementations, encouraging rigorous benchmarking and
134 cross-validation in applied research domains.

135 The unique contributions can be summarized as follows:

- 136 1. **Full pipeline serialization.** Rather than serializing only model weights, ModularML serializes
137 data definitions with assigned unique identifiers, split logic, sampling configuration, graph
138 topology, per-phase loss routing, and training and evaluation configurations. This enables
139 complete auditability of published ML studies, as shown in [Figure 2](#).
- 140 2. **Backend-agnostic DAG modeling.** By supporting backend-agnostic modeling, ModularML
141 extends adoption of existing ML libraries rather than replacing them, reducing friction
142 between works that use different packages.
- 143 3. **Declarative syntax.** All aspects of an ML experiment are constructed via declarative,
144 configuration-driven objects. This makes experiment definitions explicit, simplifies review,
145 and supports systematic iteration across training techniques and model architectures.
- 146 4. **Built-in visualization for validation.** Visual summaries help detect data leakage,
147 misconfigured sampling, incorrect loss routing, unexpected model topologies, and
148 training sequencing issues ([Figure 2](#)). This enables validation at definition time,
149 ensures execution matches intent, and lowers the barrier for domain scientists to review
150 ML-based results.

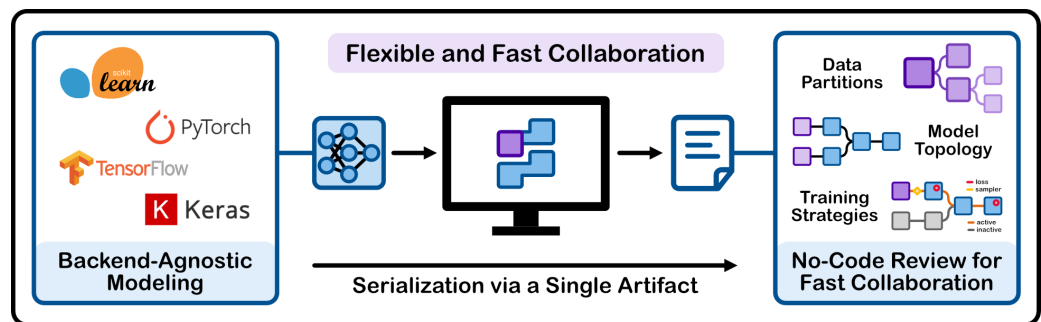


Figure 2: Backend-agnostic and fully serializable experiment workflow in ModularML with built-in visualization utilities.

Mathematics

ModularML does not introduce new mathematical formulations; instead, it codifies well-established training loops, loss aggregation, and optimizer steps across common ML backends. Any model- or loss-specific mathematics is delegated to user-defined modules or external libraries, ensuring that ModularML remains an orchestration layer that provides continued support for existing ML packages.

AI usage disclosure

Generative AI tools were used in a limited capacity during the development of this software, confined to helping structure docstrings and scaffold unit tests. All AI-assisted contributions were reviewed and verified by the authors. AI was not used in the writing of this manuscript.

Acknowledgements

We thank the ModularML contributor community for feature ideas, bug reports, and documentation improvements, and acknowledge Professor Ryan Cooper for guidance in navigating the open-source environment.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. <https://www.tensorflow.org/>
- Arrow. (n.d.). In *GitHub repository*. GitHub. <https://github.com/apache/arrow>
- Chollet, F., & others. (2015). *Keras*. <https://keras.io>
- Fomin, V., Anmol, J., Desroziers, S., Kriss, J., & Tejani, A. (2020). High-level library to help with training neural networks in PyTorch. In *GitHub repository*. GitHub. <https://github.com/pytorch/ignite>
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

- 180 Howard, J., & others. (2018). *Fastai*. GitHub. <https://github.com/fastai/fastai>
- 181 Kolesnikov, S. (2018). Catalyst - accelerated deep learning r&d. In *GitHub repository*. GitHub.
182 <https://github.com/catalyst-team/catalyst>
- 183 Lhoest, Q., Villanova del Moral, A., Platen, P. von, Wolf, T., Šaško, M., Jernite, Y., Thakur, A.,
184 Tunstall, L., Patil, S., Drame, M., Chaumond, J., Plu, J., Davison, J., Brandeis, S., Sanh,
185 V., Le Scao, T., Canwen Xu, K., Patry, N., Liu, S., ... Delangue, C. (2021). Datasets: A
186 Community Library for Natural Language Processing. *Proceedings of the 2021 Conference*
187 *on Empirical Methods in Natural Language Processing: System Demonstrations*, 175–184.
188 <https://aclanthology.org/2021.emnlp-demo.21>
- 189 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,
190 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,
191 Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python.
192 *Journal of Machine Learning Research*, 12, 2825–2830.
- 193 team, T. pandas development. (2020). *Pandas-dev/pandas: pandas (latest)*. Zenodo.
194 <https://doi.org/10.5281/zenodo.3509134>
- 195 W. Falcon, & The PyTorch Lightning team. (2019). PyTorch lightning (version 1.4). In
196 *GitHub Repository*. GitHub. <https://doi.org/10.5281/zenodo.3828935>
- 197 Wandb. (n.d.). In *GitHub repository*. GitHub. <https://github.com/wandb/wandb>
- 198 Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching,
199 S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating
200 the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41, 39–45. <https://api.semanticscholar.org/CorpusID:83459546>
201